

Your students already understand React, so we'll focus on **connecting React to the server**, **fetching JSON data**, and **displaying it cleanly**.

CLASS NOTE: Testing a Node.js JSON Server with Vite + React

Class Objectives

By the end of this class, students will:

1. Create a **React frontend** using **Vite**.
2. Connect it to the **Node.js/Express JSON API**.
3. Fetch and display data using the **Fetch API**.
4. Handle **loading** and **error** states properly.

Module 1: Setting Up the React Project

Keywords

- **Vite** → A fast frontend build tool for React and other frameworks.
- **Fetch API** → Used to make network requests from the browser.
- **CORS (Cross-Origin Resource Sharing)** → A browser security rule that restricts requests between different origins.

Step 1: Create a New Vite React App

In your terminal:

```
npm create vite@latest react-client -- --template react
```

Then navigate to the folder:

```
cd react-client  
npm install
```

⚙️ Step 2: Start the Development Server

```
npm run dev
```

✅ You should see your app running on

👉 <http://localhost:5173>

🧩 Module 2: Connecting React to the Node.js Server

We assume your **Node.js/Express server** is running at:

👉 <http://localhost:3000>

🔧 Step 1: Enable CORS on your server (if not yet done)

In your Node or Express server, install and use CORS:

```
npm install cors
```

Then modify your `server.js` :

```
const cors = require("cors");  
app.use(cors());
```

This allows your React app to fetch data from the backend.

Module 3: Fetching Data from the Server

Keywords

- **useEffect()** → Runs code after component loads (for fetching data).
- **useState()** → Stores fetched data or loading states.
- **map()** → Loops through an array to render data dynamically.

Example Code

Edit `src/App.jsx` :

```
import { useEffect, useState } from "react";

function App() {
  const [students, setStudents] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    fetch("http://localhost:3000/students")
      .then((res) => {
        if (!res.ok) throw new Error("Network response was not ok");
        return res.json();
      })
      .then((data) => {
        setStudents(data);
        setLoading(false);
      })
      .catch((err) => {
        setError(err.message);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Loading students...</p>;
  if (error) return <p>Error: {error}</p>;

  return (
    <div style={{ padding: "20px", fontFamily: "Arial" }}>
      <h1>📖 Student List</h1>
      <ul>
        {students.map((student) => (
          <li key={student.id}>
            <strong>{student.name}</strong> – {student.course}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

Code Explanation

Part	Meaning
<code>useEffect()</code>	Runs once when the component loads to fetch data.
<code>fetch()</code>	Makes an HTTP request to your Node.js API.
<code>.then(res => res.json())</code>	Converts response to JSON.
<code>useState()</code>	Stores students, loading, and error states.
<code>.map()</code>	Loops over the array to display each student.

Module 4: Testing It

Steps

1. Start your **backend server**

```
node server.js
```

It should be running at <http://localhost:3000>

2. Start your **React app**

```
npm run dev
```

It runs on <http://localhost:5173>

3. Visit your React app in the browser —

You should see a **list of students** fetched from your server 

Module 5: Handling Multiple Routes

Try extending your React app to show different data sets.

Task 1

Fetch data from `/info` and display app details.

```
useEffect(() => {
  fetch("http://localhost:3000/info")
    .then((res) => res.json())
    .then((data) => console.log(data));
}, []);
```

Then show it in your React UI.

 Task 2

Add a button that refreshes the students list manually:

```
<button onClick={() => window.location.reload()}>Refresh</button>
```

 Module 6: Common Errors & Fixes

Error Message	Cause	Solution
CORS error	Browser blocked request from different origin	Install and use cors in Node.js
Unexpected token < in JSON	Server returned HTML, not JSON	Ensure res.json() or JSON.stringify() used
Network Error	Server not running	Start Node.js server on port 3000
students.map is not a function	Server returned an object, not an array	Check the server response in console

 Module 7: Summary

Concept	Purpose	Example
fetch()	Get data from API	fetch('http://localhost:3000/students')
useEffect()	Fetch data after component loads	useEffect(() => {...}, [])

Concept	Purpose	Example
useState()	Manage app state	<code>const [students, setStudents] = useState([])</code>
map()	Render list items	<code>students.map(s => {s.name})</code>
CORS	Allows frontend-backend connection	<code>app.use(cors())</code>

Class Discussion

1. Why do we need **CORS** when connecting React to Node.js?
2. What happens if the **backend server** is not running?
3. How can you make your frontend dynamically update when new data is added to the backend?

Bonus: Add Styling (Optional)

You can add a simple style using Tailwind or inline styles.

Example with inline CSS:

```
<div style={{  
  background: '#f5f5f5',  
  padding: '20px',  
  borderRadius: '10px',  
  width: '60%',  
  margin: '20px auto'  
}}>
```